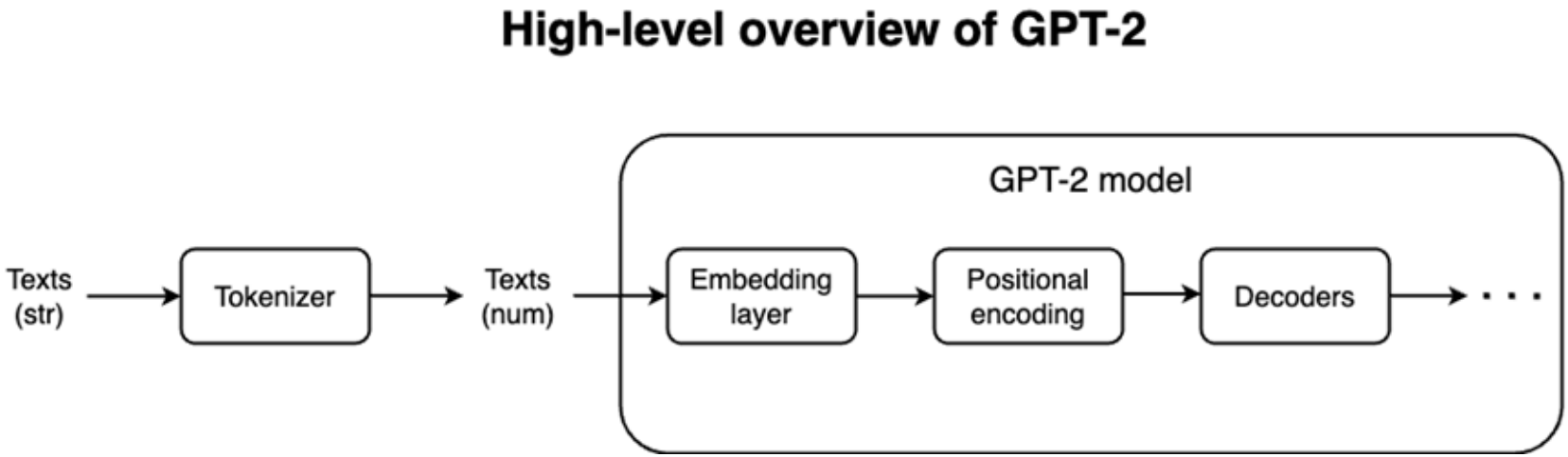
LECTURE 3: Introduction to LLMs

Zhao Zhang
Rutgers University

Overview

1. High-level overview of the workflow of GPT2 model.
2. Tokenization
3. Layers in GPT-2 model
    a. Embedding layer
    b. Positional encoding layer
    c. Decoder layer
        i. Layernorm
        ii. Multi-head self-attention mechanism
        iii. MLP (or feedforward layer)

High-level overview



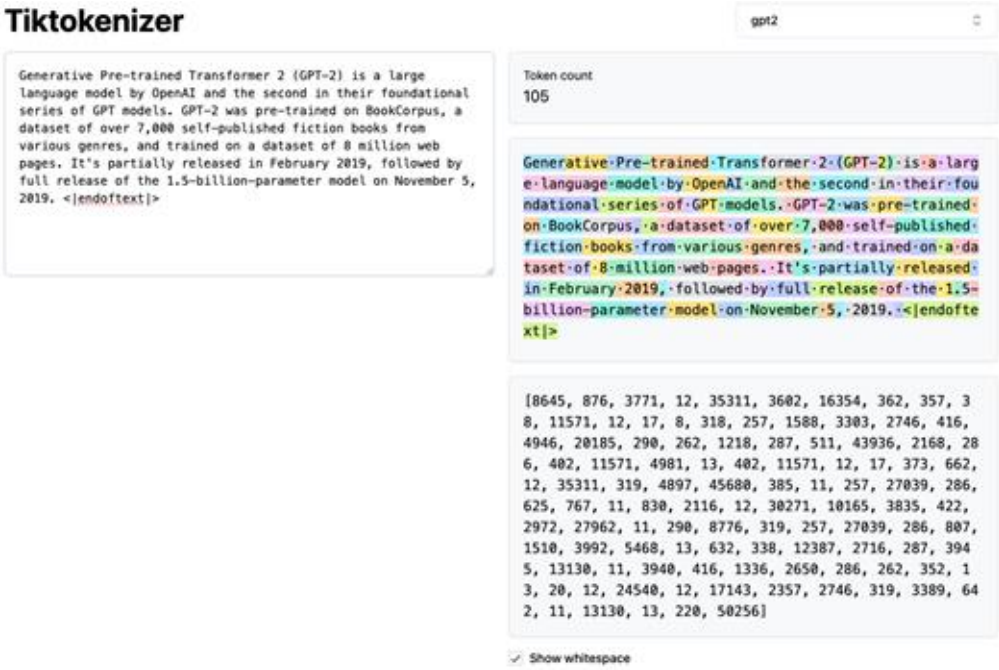
Tokenization

Tokenizer

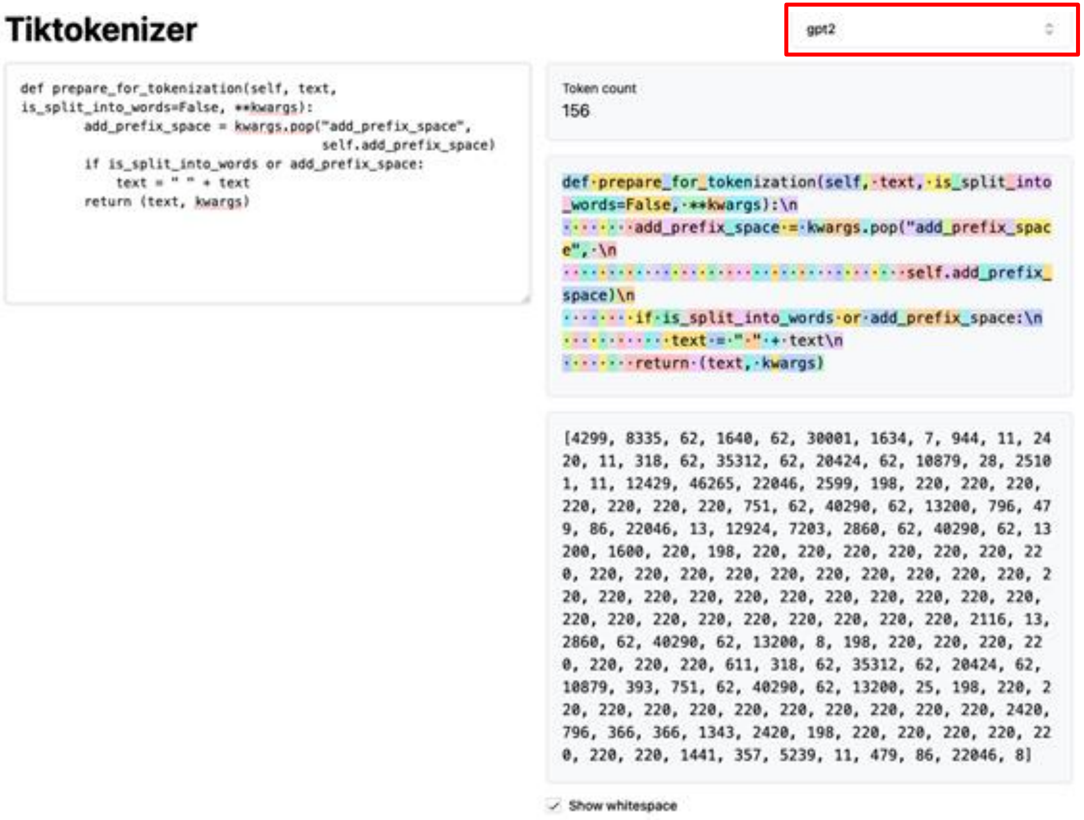
- Why do we need a tokenizer?
  - ML/DL can only handle matrices/tensors.
  - A string of words need to be converted into numerical values by Tokenizer.
- Tokenizer is not part of the GPT-2 model.
  - **But it is very important!**
  - It determines how the model sees the strings, which can cause significant differences on model's inference performance.
- Tokenizer is trained from the pretraining dataset.
  - So, different LLMs have their own tokenizers.

Tokenization

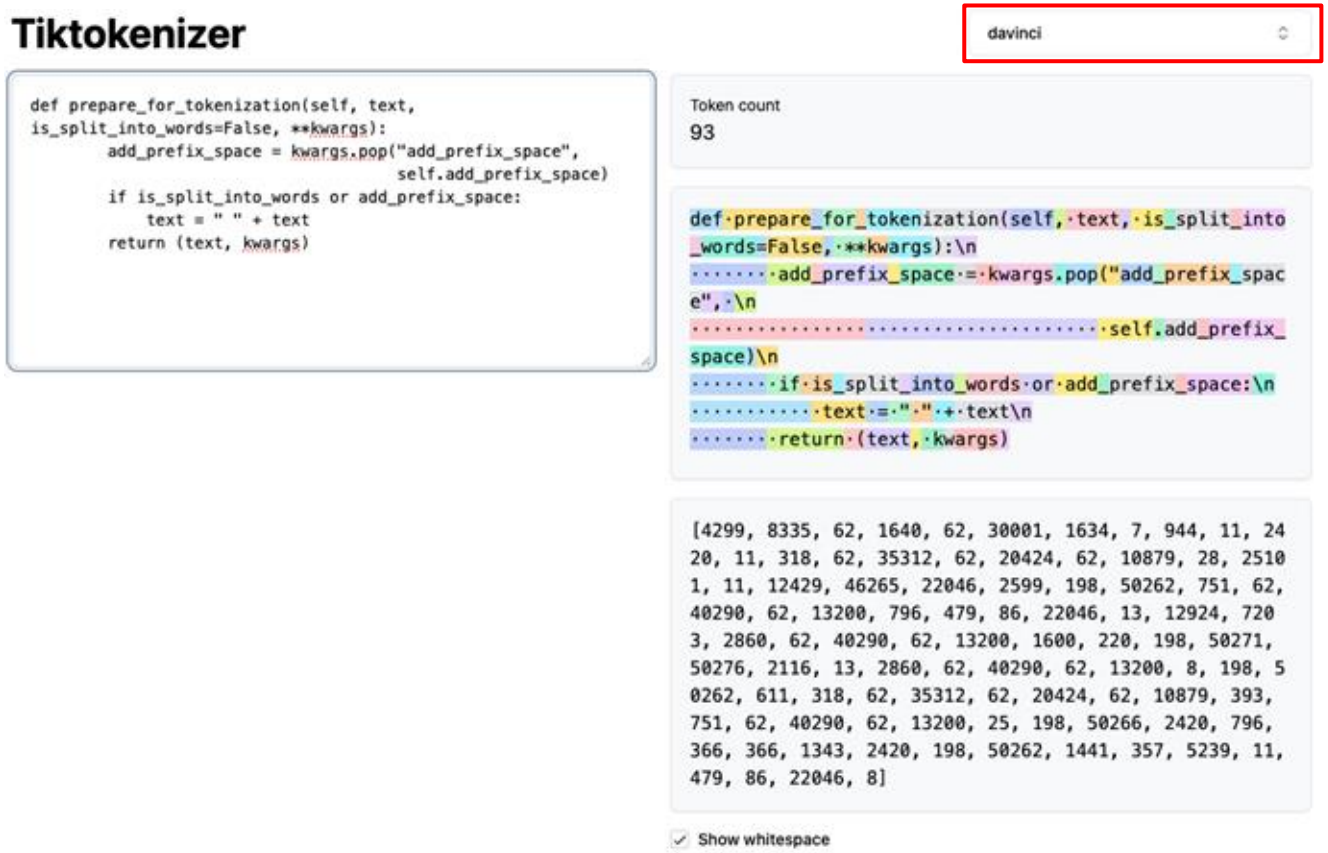
- One tokenizer has two parts
  - Encoder and decoder.
  - Encoder: a string -> numbers.
  - Decoder: numbers -> a string.
- The simplest tokenizer
  - Encode chars to UTF-8 values.
  - But computationally intractable because attention mechanism has O(n^3) time complexity.
- Try different tokenizers: link



GPT-2 tokenizer vs GPT-3 tokenizer



GPT-2 tokenizer vs GPT-3 tokenizer



GPT-2's tokenization strategy

- **Byte Pair Encoding (BPE)** (source code).
- Continues merging pairs of UTF-8 encoded bytes.
- Here is an example:
  - Suppose we have already converted every char in a string into its byte representation.
    - For example, "aaabdaaabac" -> [97, 97, 97, 98, 100, 97, 97, 97, 98, 97, 99].
  - Then we group every char with its right adjacent char.
    - [(aa),(aa),(ab),(bd),(da),(aa),(aa),(ab),(ba),(ac)]
    - [(97,97),(97,97),(97,98),(98,100),(100,97),(97,97),(97,97),(97,98),(98,97),(97,99)]
  - We find the most frequent pair, which in this case, is (aa).
  - Every (aa) pair is replaced by a new special char, say "Z".
    - "aaabdaaabac" -> "ZabdZabac".
    - We give index 256 (UTF-8 encoded value ranges from 0 to 255) to "Z" and store this info into a lookup table.
  - Then we continue the steps above with the new str.
    - Every step will create a new index, and we can choose when to stop.

GPT-2 tokenizer details

- GPT-2 tokenizer works at the word-level, not the string-level.
  - I.e. OpenAI team first uses regex operations to break a string into many word-like chunks, and then perform encoding on these chunks.

```
# Should haved added re.IGNORECASE so BPE merges can happen for capitalized versions of contractions
self.pat = re.compile(r"""'s|'t|'re|'ve|'m|'ll|'d| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+""")
```

  - These are chunks instead of words because 1) there are usually numbers in the text, 2) if a word has a leading white space, that white space is grouped with the word (e.g. “hello world” -> “hello”, “ world”), 3) punctuations are usually separated from word, and 4) “I’ve” -> (“I”,“’ve”.
- Ran BPE algo for 50,000 iterations on WebText creates a lookup table of 50,256 items.
  - One more special token as the 50,257th item, <|endoftext|>, which separates two contexts.
- To check its lookup table: link.

GPT-2 tokenizer encoding

1. Use regex to break a string into chunks.
   a. “Hello World!” -> [“Hello”, “ World”, “!”]
2. For each chunk, encode it to UTF-8 format.
   a. “Hello” -> [\x48, \x65, \x6C, \x6C, \x6F]
   b. “ World” -> [\x20, \x57, \x6F, \x72, \x6C, \x64]
   c. “!” -> [\x21]
3. Use the lookup table to iteratively compress tokens, starting with pair that has the smallest value first.
   a. [“H”, “e”, “l”, “l”, “o”] -> Try pair w/ the adjacent right and find the one has smallest value in Text Bytes.
4. Add the special token <|endoftext|> to the end of a text.
5. Finally, put all together to form a tensor of tokens.
6. Note that these all happen before the pretraining.
   a. This is what “megatron-lm/tools/preprocess_data.py” does.

GPT-2 tokenizer decoding

- Decoding is needed when we want to generate a sentence.
- It is the reverse of encoding.
- Given a list of tokens (i.e. integers), we need to convert them into a string.
  - Also an iterative process.
  - Decompose one token into a pair of tokens until every token cannot be decomposed anymore.
  - Finally, use the inverse of the lookup table to convert each token to a char.

Layers in GPT-2 model

How to construct the pretrianing as an unsupervised learning problem?

Consider pre-training as an optimization problem

- GPT-2 was trained using unsupervised learning method.
  - No manually curated labels needed.
- Given an unsupervised corpus of tokens U = {u_1, …, u_n}, we use a standard language modeling objective to maximize the following likelihood:

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \ldots, u_{i-1}; \Theta)$$

  Where *k* is the context window, and *P* is modeled using a neural network w/ parameters Θ.

Another way to understand the loss func.

- To get the probability of generating a target “sentence”, we multiply the conditional probability of generating each target “word”:

$$P(U) = \prod_i P(u_i | u_{i-k}, \ldots, u_{i-1}; \Theta)$$

- Take the log of the a product and make it a summation problem (w/ same minimization target):

$$\begin{aligned} \log(P(U)) &= \log \left(\prod_i P(u_i | u_{i-k}, \ldots, u_{i-1}; \Theta) \right) \\ &= \sum_i \log P(u_i | u_{i-k}, \ldots, u_{i-1}; \Theta) \end{aligned}$$

- log(p) can be calculated from cross entropy loss (torch.nn.CrossEntropyLoss).

How to generate probability

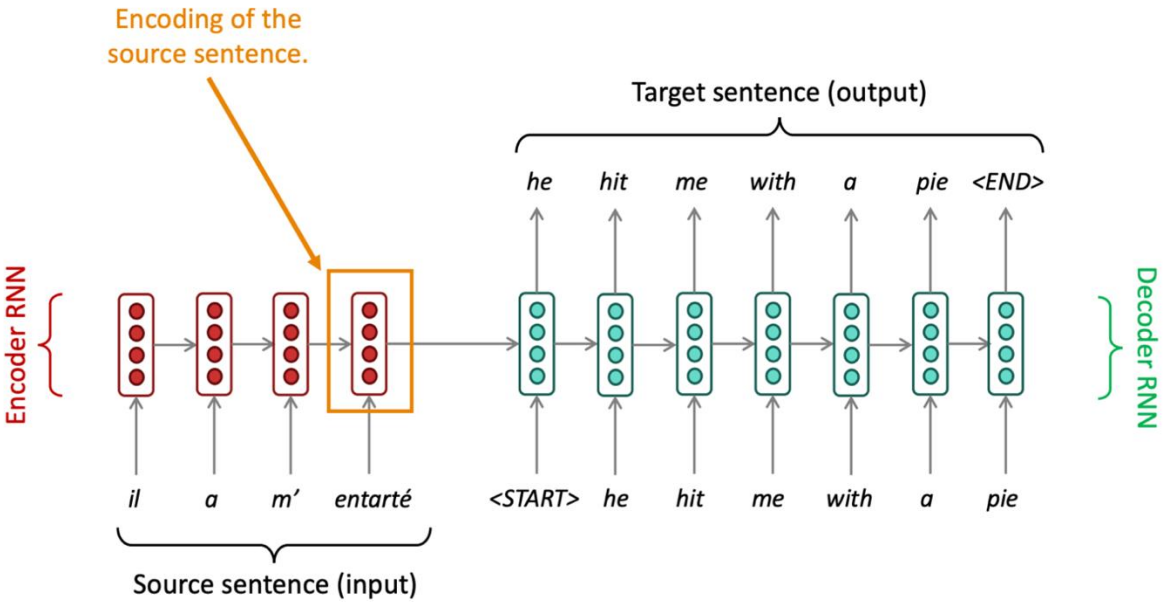
- Use layers of Transformer decoder.
- This model applies a multi-headed self-attention operation over the input context tokens followed by position-wise feedforward layers to produce an output distribution over target tokens

$$h_0 = UW_e + W_p$$
$$h_l = \texttt{transformer\_block}(h_{l-1}) \forall i \in [1, n]$$
$$P(u) = \texttt{softmax}(h_n W_e^T)$$

  where W_e is the embedding layer, W_p is the positional encoding layer, and there are *n* decoders in this model.

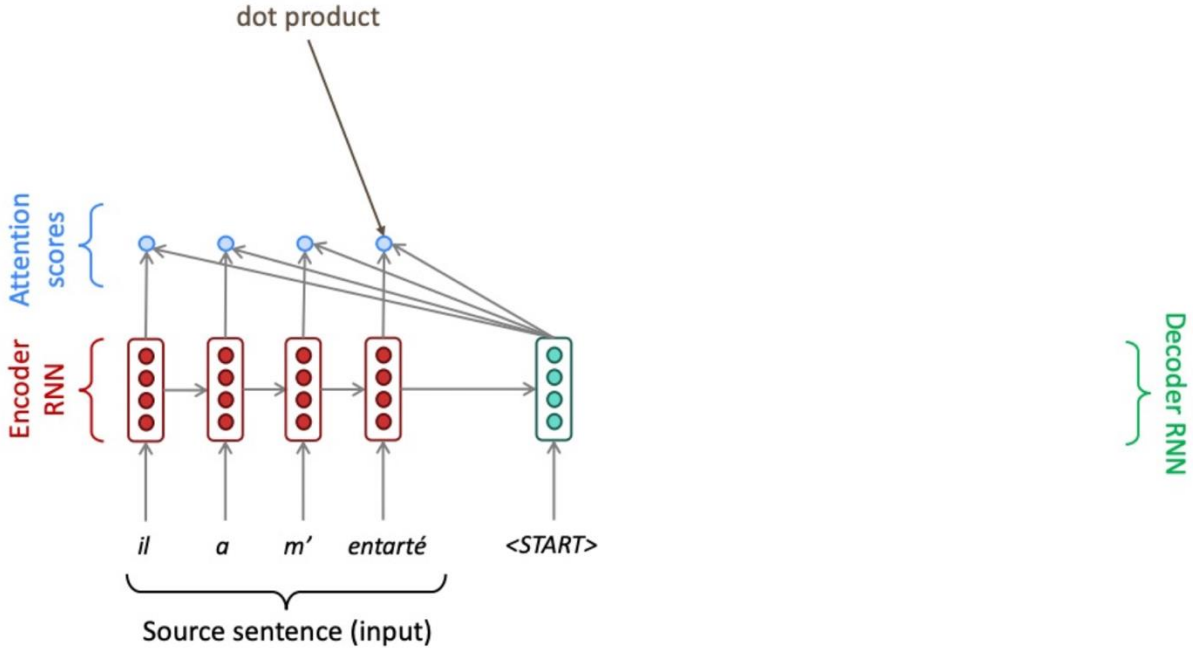
- Note that we need W_e both for the input and output.

RNN in Machine Translation



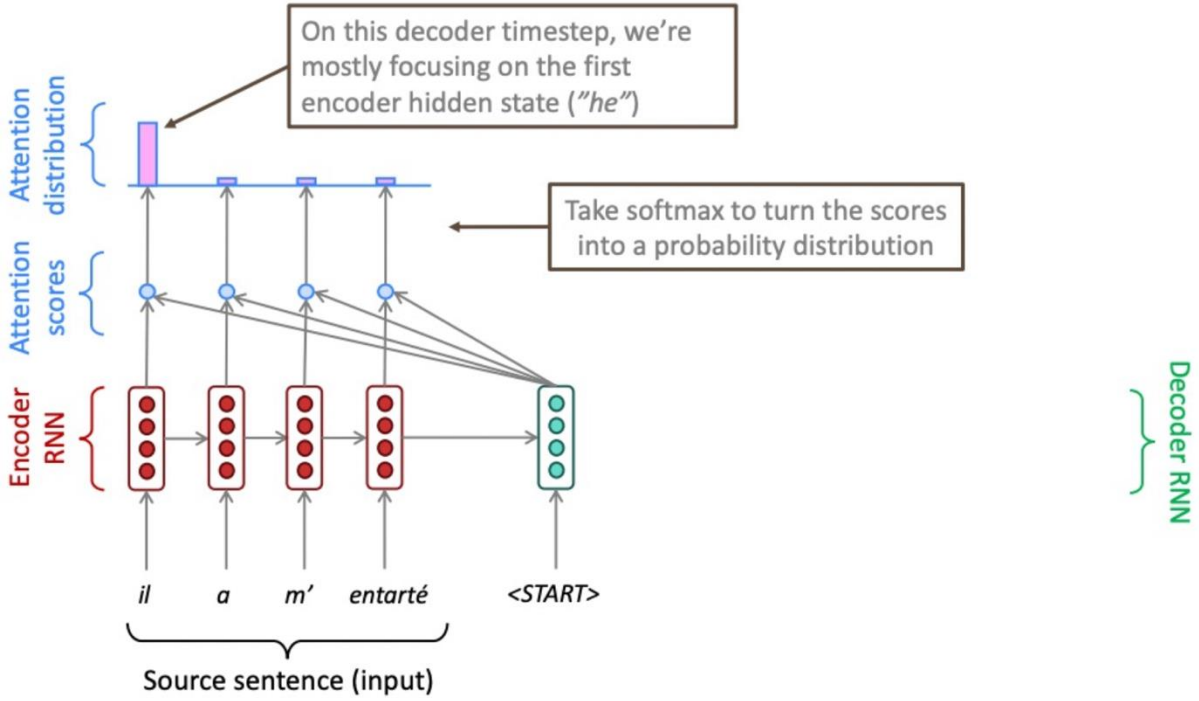
* https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1214/slides/cs224n-2021-lecture07-nmt.pdf

Sequence-to-sequence with Attention



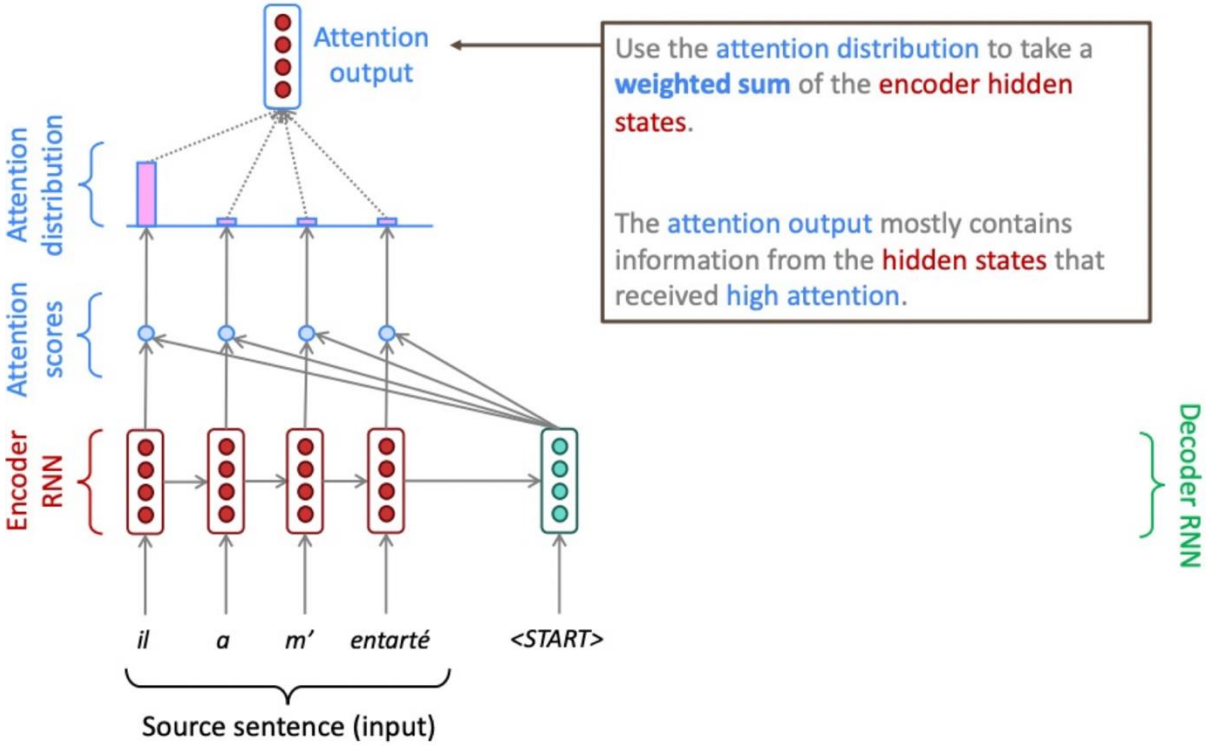
* https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1214/slides/cs224n-2021-lecture07-nmt.pdf

Sequence-to-sequence with Attention



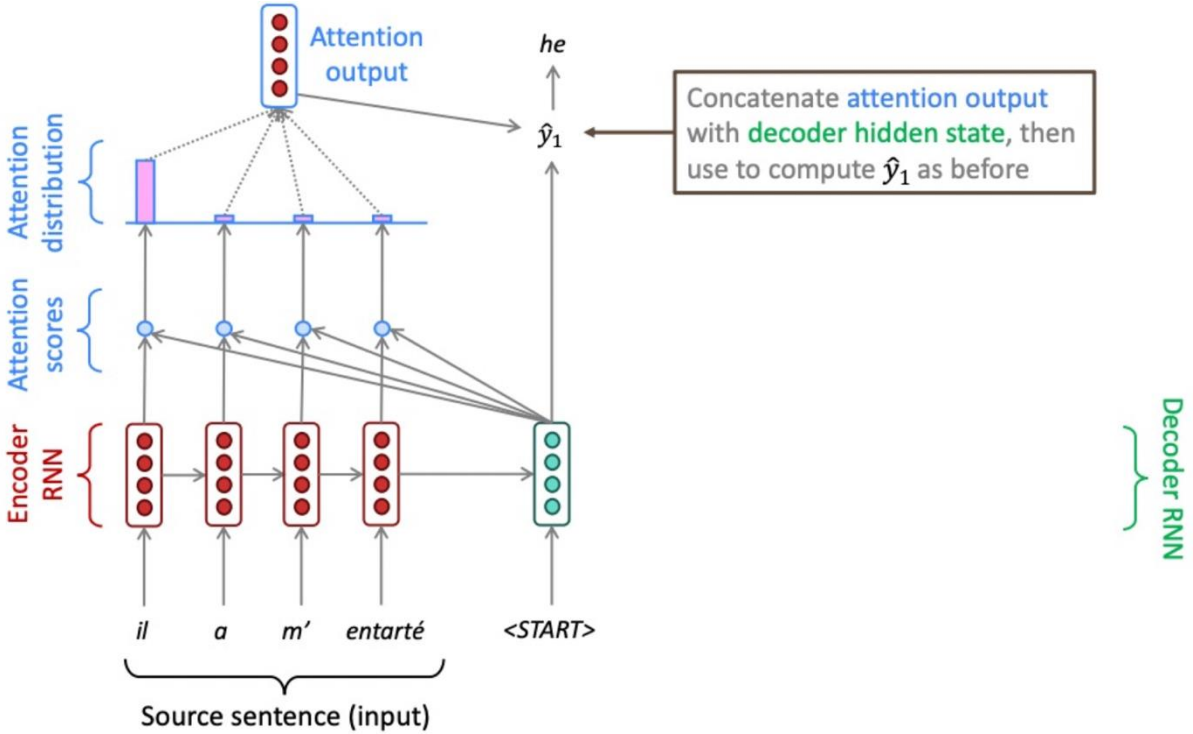
* https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1214/slides/cs224n-2021-lecture07-nmt.pdf

Sequence-to-sequence with Attention

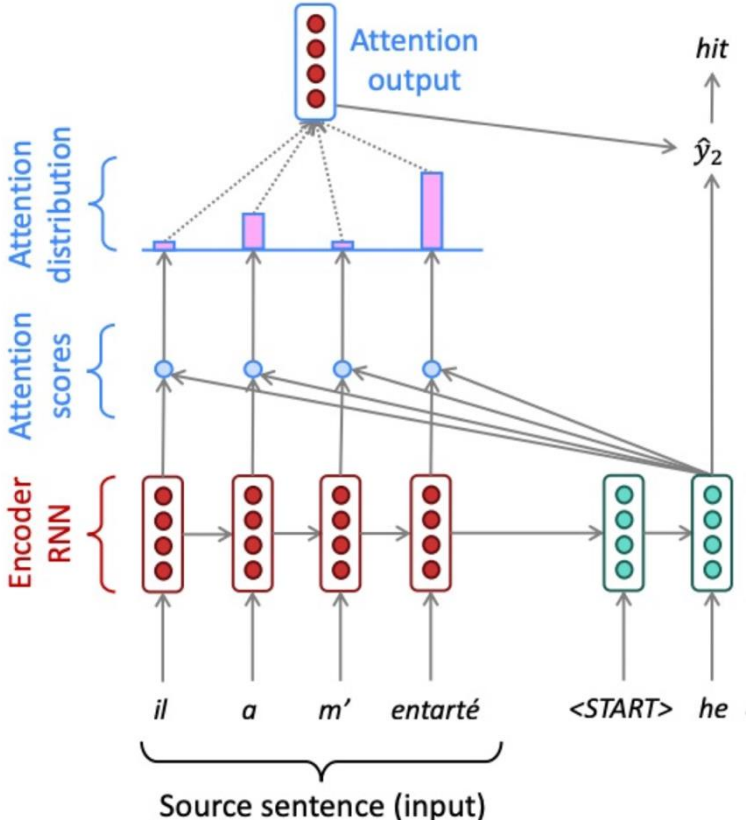


* https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1214/slides/cs224n-2021-lecture07-nmt.pdf

Sequence-to-sequence with Attention

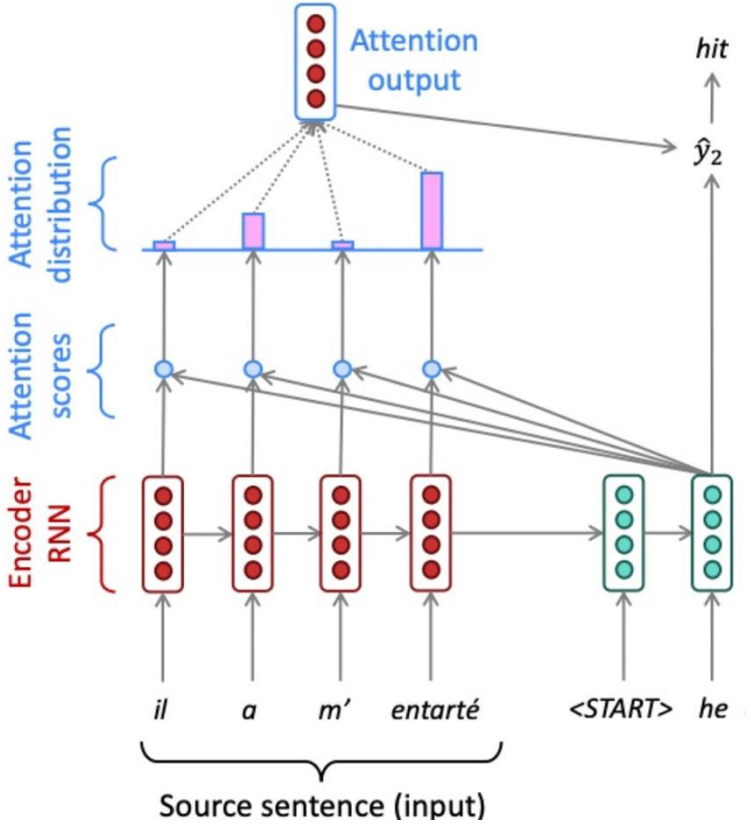


Sequence-to-sequence with Attention



- Encoder hidden states: $h_1, h_2, \ldots, h_N \in \mathbb{R}^h$
- Decoder hidden state at time t: s_t
- Attention Scores (e^t) for this step: $\boldsymbol{e^t} = [\boldsymbol{s_t^T h_1}, \ldots, \boldsymbol{s_t^T h_N}] \in \mathbb{R}^N$
- Attention weights/distribution (α^t): $\alpha^t = softmax(e^t) \in \mathbb{R}^N$
- Attention Output (a^t):
  - Weighted sum of encoder hidden states using (α^t) :

Sequence-to-sequence with Attention



- Attention Output (a^t):
  - Weighted sum of encoder hidden states using (α^t) :

$$a_t = \sum_{i=1}^{N} \alpha_i^t h_i \in \mathbb{R}^h$$

- Finally we concatenate the attention output a_t with the decoder hidden state s_t and proceed as in the non-attention seq2seq model
  - $[a_t; s_t] \in \mathbb{R}^{2h}$

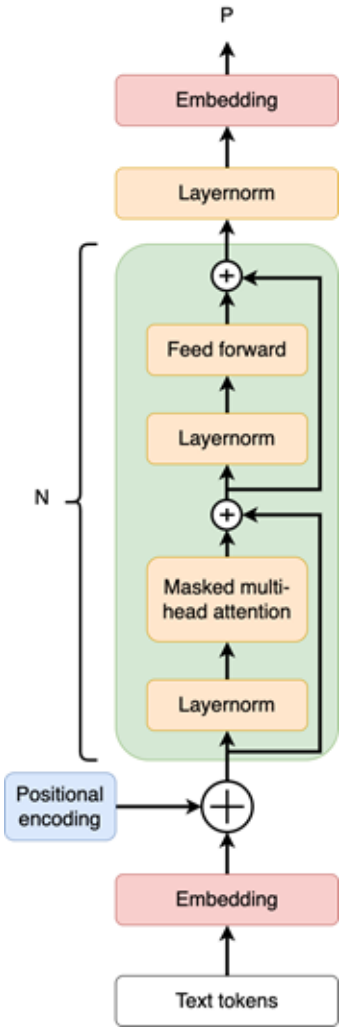
A more concrete example

- This is an oversimplified example.
- Hopefully, it can convey the basic ideas of input and output shapes.
- Assume we have a training corpus as:
  - “Generative Pre-trained Transformer 2 (GPT-2) is a large language model by OpenAI and the second in their foundational series of GPT models.”
- We first need to break a corpus into blocks/chunks:
  - Suppose we have block size (or context size *k*) of 8.
  - [“Generative Pre-trained Transformer 2 (GPT-2) is a large language”, “model by OpenAI and the second in their”, “foundational series of GPT models.”]
  - If we focus on the 1st chunk:
    - Input: [[“Generative”], [“Generative”, “ Pre-trained”], …, [“Generative”, “ Pre-trained”, “ Transformer”, “ 2”, “ (“, “GPT”, “-”, “2”, “)”, “ is”, “ a”, “ large”, “ language”]]
    - Output: [“Pre-trained”, “Transformer”, …, “model”].
    - Output is simply the next word of the input words in their original corpus.

Model architecture

GPT-2's block diagram

- GPT-2 model is similar to GPT-1.
- Except some corrections:
  - Layer norm is moved to the place before attention and MLP block.
  - An additional layer norm is added to the end of decoder layers.
- We will cover **embedding layer**, **positional encoding layer**, **multihead attention block**, and **layer norm** in more details in the following slides.
- Code can be found here.



Pre-layernorm residual

Code snippet

- A very simple implementation of the decoder layer (link)

```
class Block(nn.Module):
    def __init__(self, n_ctx, config, scale=False):
        super(Block, self).__init__()
        nx = config.n_embd
        self.ln_1 = LayerNorm(nx, eps=config.layer_norm_epsilon)
        self.attn = Attention(nx, n_ctx, config, scale)
        self.ln_2 = LayerNorm(nx, eps=config.layer_norm_epsilon)
        self.mlp = MLP(4 * nx, config)

    def forward(self, x, layer_past=None):
        a, present = self.attn(self.ln_1(x), layer_past=layer_past)
        x = x + a
        m = self.mlp(self.ln_2(x))
        x = x + m
        return x, present
```

Embedding layer

After tokenization…

- After converting strings to tokens, can we directly use these tokens for the attention mechanism?
- No! We need to represent each token by a vector.
- Why bother? If two “words” have similar semantic meaning (e.g. “good” and “nice”), we also want them to have similar mathematical representations (maybe in a higher dimensional space).
- To know more about it, check the distributed representation of words paper.

Example of vector representations of words

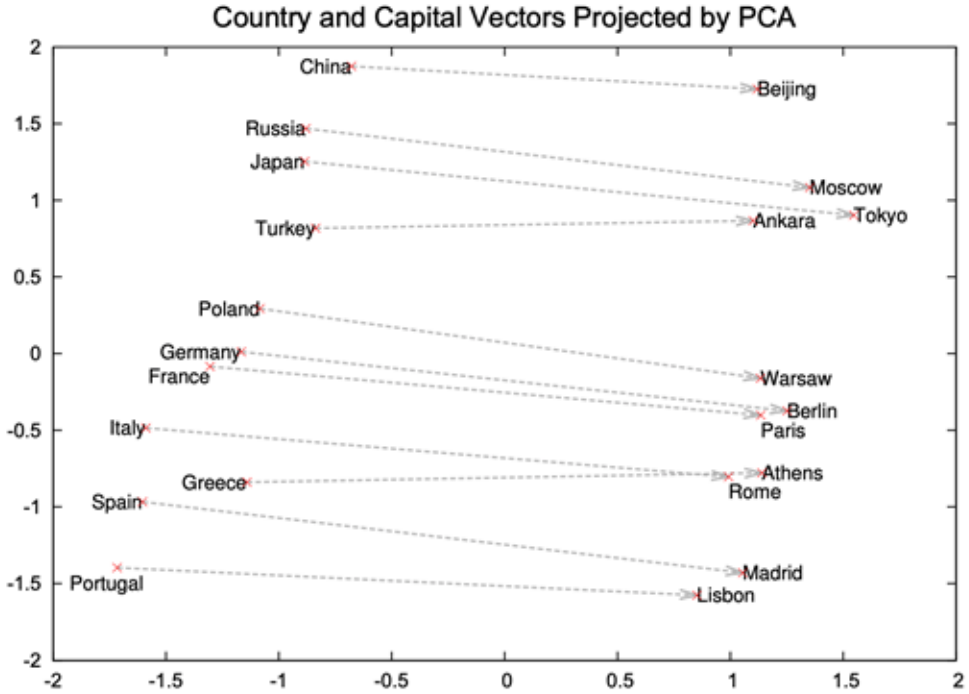
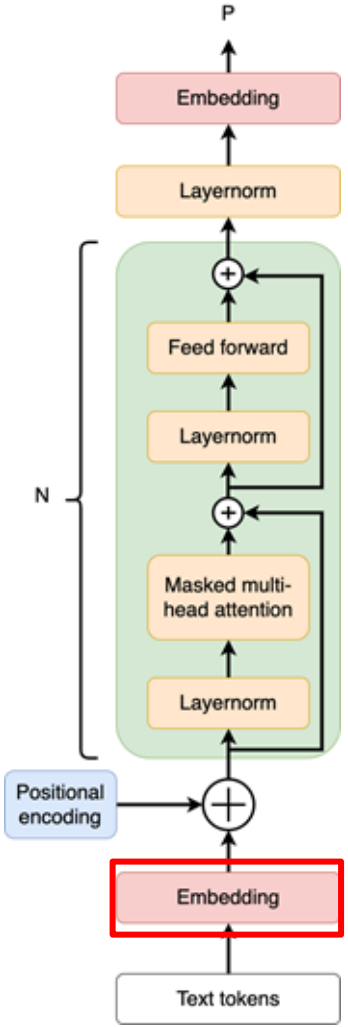


Figure 2: Two-dimensional PCA projection of the 1000-dimensional Skip-gram vectors of countries and their capital cities. The figure illustrates ability of the model to automatically organize concepts and learn implicitly the relationships between them, as during the training we did not provide any supervised information about what a capital city means.

Embedding layer

- How to convert tokens to vectors?
- Use embedding layer!
  - Embedding layer is a lookup table (or a dict in Python).
  - Tokens are the keys, and vectors are the values.
- **Embedding layer is trainable**.
- Check torch.nn.Embedding to see examples and implementation details.



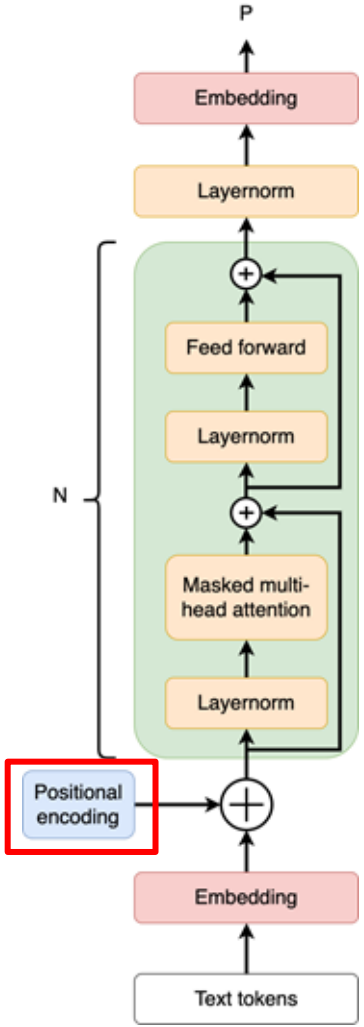
Padding

- From the example in page 18, we see that the number tokens in each sequence is not the same.
- This is problematic for matrix operations (we will see later in the self-attention section).
- Therefore, padding is needed.
- Pad to the right by a special “padding token”.
- However, this brings up other problems when we talk about self-attention and normalization.

Positional encoding layer

Positional encoding layer

- After the embedding layer, we need a positional encoding layer.
- Why? Because we want to let model have the positional info of “words” in the string, so that we can perform operations on them in parallel.
- There are many ways to construct the positional encoding layer, for example, in the transformer paper, they use cos/sin function.
- In GPT-2 paper, they use torch.nn.Embedding as the positional encoder, making it trainable.



Layer normalization

Why normalization?

- It provides better model performance in terms of accuracy.
  - The gradients weights in one layer are highly dependent on the outputs of the neurons in the previous layer especially if these outputs change in a highly correlated way.
  - Batchnorm alleviate this problem in the backprop process.
- The most commonly used norm is batch normalization.
  - We normalize across all the training examples in the current batch.

$$\mathrm{BatchNorm}(u) = \gamma \frac{u - \mu}{\sigma + \epsilon} + \beta$$

$$\mu = \frac{1}{B}\sum_{b=1}^{B} u_b,\ \sigma = \sqrt{\frac{1}{B}\sum_{b=1}^{B}(u_b - \mu)^2}$$

where μ and σ keep the same dim as the number of features in a batch, and γ and β are trainable parameters.

Layer normalization

- Suppose every example has d-dim

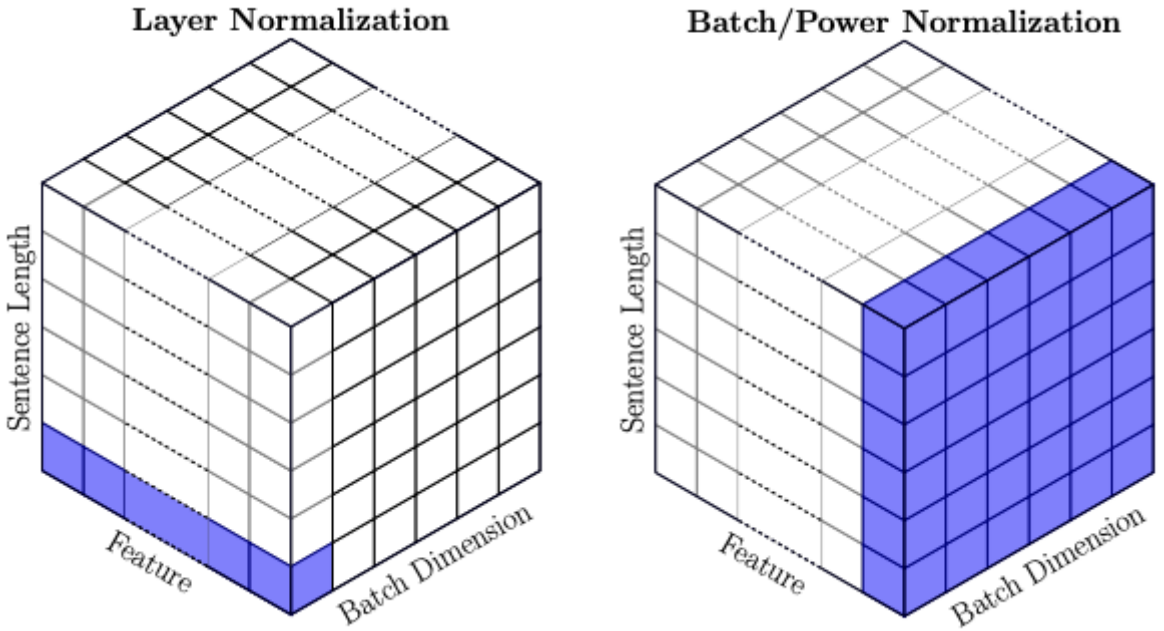
$$\text{LayerNorm}(u) = \gamma \frac{u - \mu}{\sigma + \epsilon} + \beta$$

$$\mu = \frac{1}{d}\sum_{i=1}^{d} a^i,\ \sigma = \frac{1}{d}\sqrt{\sum_{i=1}^{d}(a^i - \mu))}$$

  where μ and $\boldsymbol{\sigma}$ are 1-dim values.

- For sentences, we perform layernorm on all the features in one “word”, which is usually the last dim in K.shape, Q.shape, and V.shape.
- Torch document: link, and the paper.

Visualization



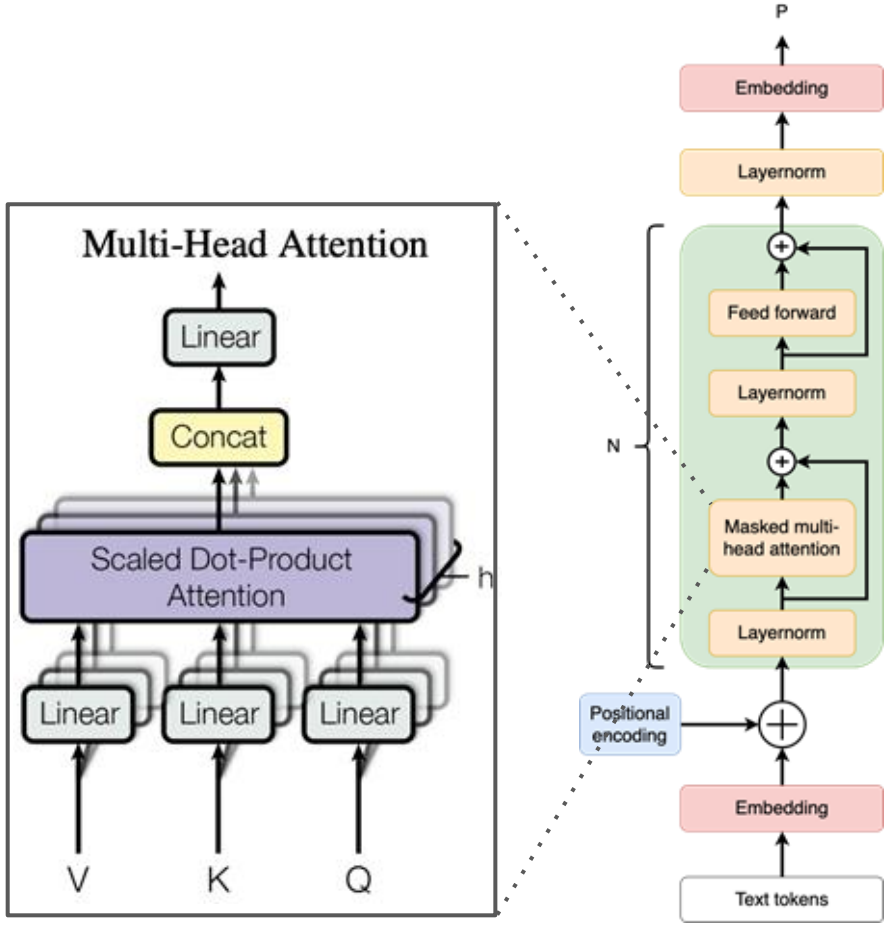
Why layer norm instead of batch norm?

- **Sentences differ in length.**
- A minibatch has sentences with different length **->** μ and σ in batch norm changes frequently across different mini-batches.
- μ and σ learned in training might not be suitable for unseenly long sentences in inference.

Multi-head self-attention mechanism

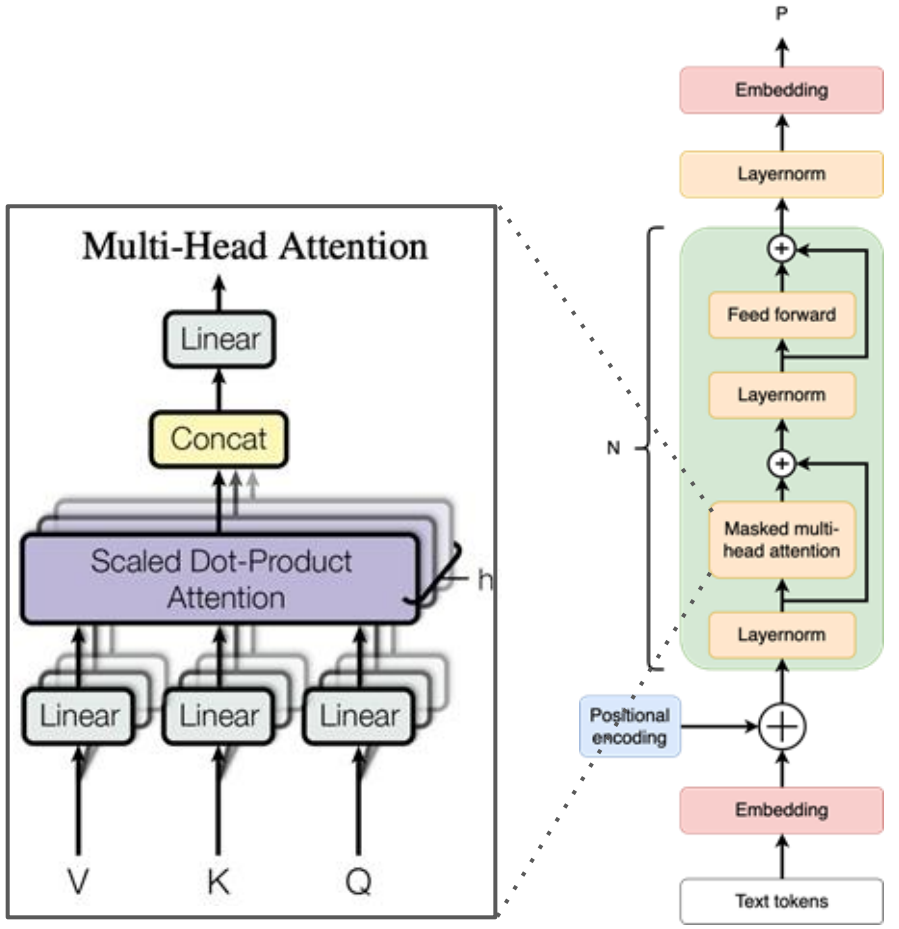
Multi-head Attention - cont.

- Key (K), Value (V), and Query (Q) are the same.
- But, after passing them through the Linear layers (feed forward layers), they will differ.
- K, V, and Q are splitted along the d_model dim, where d_model is the dim of vectors stored in embedding layers and the output dim of each decoder block.



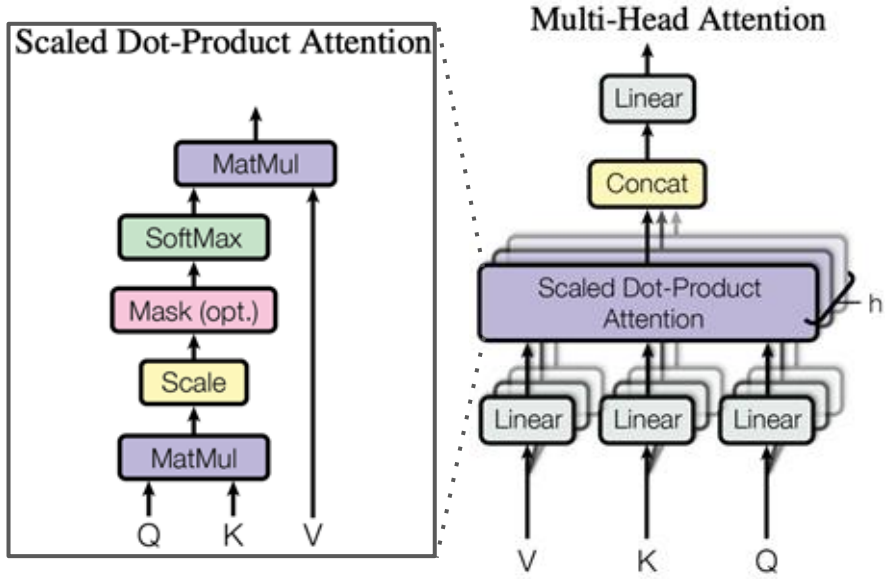
Multi-head Attention - cont.

- After the embedding layers, the input has three dim: (batch, sequence_len, d_model).
- After the split, each of them has shape: (batch, sequence_len, num_head, d_model//num_head).
- All these dims are hyper-params.
- This multi-head split can help reduce the computation in each attention block significantly.



Masked self-attention

- torch.matmul(Q,K) measures similarities between vect. in Q and K.
- Scaling by sqrt(d_model//num_head) to avoid having all prob concentrate on one entry in the matmul result.
- Mask is a lower-triangular matrix, which masks the query from future key content.
- Softmax gives the prob.
- This whole process can be seen as computing the weighted average sum of vectors in V.



$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

A more concrete example

- Let's reuse the example in page 18 and assume batchsize = 1.
  - Input: [["Generative"], ["Generative", " Pre-trained"], …, ["Generative", " Pre-trained", " Transformer", " 2", " (", "GPT", "-", "2", ")", " is", " a", " large", " language"]]
- Let's ignore the linear transformation and consider Q=K=V.
- Let's also ignore multihead attention for simplicity.
- Let's visualize Q, K, and V in matrix form:

$$Q = K = V = \begin{bmatrix} \overrightarrow{\text{Generative}} & \overrightarrow{\text{Emb}(0)} & \overrightarrow{\text{Emb}(0)} & \cdots & \overrightarrow{\text{Emb}(0)} \\ \overrightarrow{\text{Generative}} & \overrightarrow{\text{Pre-trained}} & \overrightarrow{\text{Emb}(0)} & \cdots & \overrightarrow{\text{Emb}(0)} \\ \overrightarrow{\text{Generative}} & \overrightarrow{\text{Pre-trained}} & \overrightarrow{\text{Transformer}} & \cdots & \overrightarrow{\text{Emb}(0)} \\ \vdots & & \ddots & & \\ \overrightarrow{\text{Generative}} & & \cdots & & \overrightarrow{\text{language}} \end{bmatrix}$$

Matmul of Query and Key

- For torch.matmul(Q, K^T):

$$QK^T = \begin{bmatrix} \overrightarrow{\text{Generative}} & \overrightarrow{\text{Emb(0)}} & \overrightarrow{\text{Emb(0)}} & \cdots & \overrightarrow{\text{Emb(0)}} \\ \overrightarrow{\text{Generative}} & \overrightarrow{\text{Pre-trained}} & \overrightarrow{\text{Emb(0)}} & \cdots & \overrightarrow{\text{Emb(0)}} \\ \overrightarrow{\text{Generative}} & \overrightarrow{\text{Pre-trained}} & \overrightarrow{\text{Transformer}} & \cdots & \overrightarrow{\text{Emb(0)}} \\ \vdots & & \ddots & & \\ \overrightarrow{\text{Generative}} & & \cdots & & \overrightarrow{\text{language}} \end{bmatrix} \begin{bmatrix} \overrightarrow{\text{Generative}}^T & \overrightarrow{\text{Generative}}^T & \overrightarrow{\text{Generative}}^T & \cdots & \overrightarrow{\text{Generative}}^T \\ \overrightarrow{\text{Emb(0)}}^T & \overrightarrow{\text{Pre-trained}}^T & \overrightarrow{\text{Pre-trained}}^T & \cdots & \\ \overrightarrow{\text{Emb(0)}}^T & \overrightarrow{\text{Emb(0)}}^T & \overrightarrow{\text{Transformer}}^T & & \vdots \\ \vdots & \vdots & \vdots & \ddots & \\ \overrightarrow{\text{Emb(0)}}^T & \overrightarrow{\text{Emb(0)}}^T & \overrightarrow{\text{Emb(0)}}^T & \cdots & \overrightarrow{\text{language}}^T \end{bmatrix}$$

where \vec{Emb(0)} represents the embedding results of the “padding token”.

Need an attention mask

- Without an attention mask, we will have an attention score from the “future”:

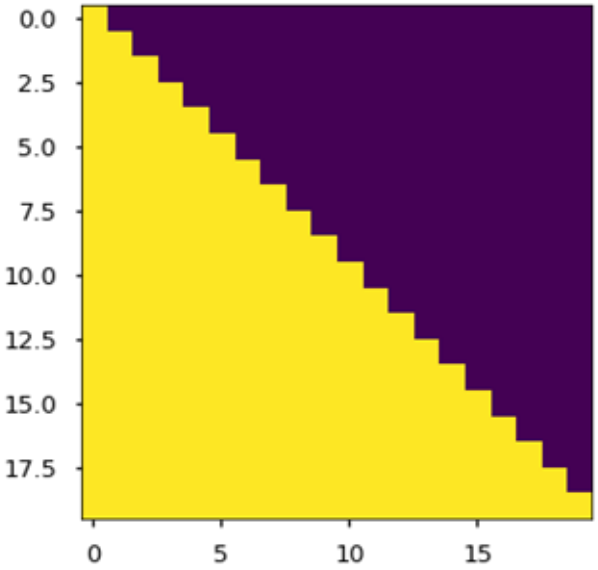
$$Q[0,:]K^T[:,1] = \left[\overrightarrow{\text{Generative}}\ \overrightarrow{\text{Emb}(0)}\ \overrightarrow{\text{Emb}(0)}\ \cdots\ \overrightarrow{\text{Emb}(0)}\right] \begin{bmatrix} \overrightarrow{\text{Generative}}^T \\ \overrightarrow{\text{Pre-trained}}^T \\ \overrightarrow{\text{Emb}(0)}^T \\ \vdots \\ \overrightarrow{\text{Emb}(0)}^T \end{bmatrix}$$

- We want the model to guess the next “word” given only the existing context.
- In other words, vectors in K and Q should not see the answers in the future. Otherwise, it is cheating!
- So, we need an attention mask after K^T \times Q, which should filter out all the attention scores computed using future information.

Attention mask

- An attention mask can be as simple as a lower triangular matrix, where the lower entries are 1s and the upper entries are 0s.
- Code snippet (link)

```
max_positions = config.max_position_embeddings
self.register_buffer(
    "bias",
    torch.tril(torch.ones((max_positions, max_positions), dtype=torch.bool)).view(
        1, 1, max_positions, max_positions
    ),
    persistent=False,
)
```



Yellow=1, purple=0

Some final reminders on MLP block

- The output from multi-head blocks are concatenated together.
- The activation they used in GPT-2 is GeLU instead of ReLU.
  - GeLU is continuous and differentiable everywhere.

```
def gelu(x):
    return 0.5 * x * (1 + torch.tanh(math.sqrt(2 / math.pi) * (x + 0.044715 * torch.pow(x, 3))))
```

- Two layers in MLP block:
  - Hidden size = 4 x d_model.
  - Output size = d_model.
- After the MLP layer, the output will have the same shape as the input, so that we can easily pass it to the next decoder layer without worrying about dims.

References

- Megatron-LM: https://github.com/NVIDIA/Megatron-LM
- Huggingface GPT-2: https://github.com/huggingface/transformers/blob/main/src/transformers/models/gpt2/modeling_gpt2.py
- PyTorch version of OpenAI's GPT-2: https://github.com/graykode/gpt-2-Pytorch/blob/master/GPT2/model.py
- Illustration of GPT-2: https://jalammar.github.io/illustrated-gpt2/